

Civic.ai RAG Thesis P2 Supervisor Briefing

Project: Civic.ai: A Bangla-English Retrieval-Augmented Generation System for Bangladeshi Government Services **Current P2 domain:** Birth and Death Registration **Prepared for:** Supervisor discussion, P2 progress update, and demo readiness **Repository branch:** `feature/add-birth-death-domain` **Latest experiment commit:** `a1f5680` **Fallback checkpoint before P2 evaluation framework:** `01c663e`

1. Executive Summary

The goal of this thesis is to build a high-accuracy, evidence-grounded, bilingual Bangla-English RAG chatbot for Bangladeshi government-service information. In the current phase, we narrowed the implementation to one active domain: **birth and death registration**. This allows us to demonstrate a complete, testable pipeline before adding other domains such as passport, BRTA, birth certificate expansion, death certificate expansion, or other civic services.

For P2, the main objective is not only to show a working chatbot. The university objective asks for:

- A design process or methodology overview.
- Multiple alternative engineering/theoretical solutions.
- Preliminary model/system specification.
- Functional verification or simulation of alternatives.
- Evidence that design choices are made under requirements and constraints.

To satisfy this, we implemented and evaluated multiple retrieval designs:

1. **Dense-only retrieval** using BGE-M3 embeddings and ChromaDB.
2. **BM25-only retrieval** using sparse lexical matching.
3. **Hybrid RRF retrieval** combining Dense and BM25 rankings using Reciprocal Rank Fusion.
4. **Hybrid + Rerank retrieval** using the fused candidate set and a reranking stage.

We also prepared the framework to compare:

- **Simple RAG vs CivicRAG.**
- **LLM backends:** `llama3.2`, `llama3`, and `qwen2.5:7b`.
- **Stability:** repeated query consistency.
- **Parameter sensitivity:** RRF weights and top-k values.

The first 20-question fast retrieval run showed:

Retrieval Variant	Recall@1	Recall@3	Recall@5	Context Precision@5	MRR	nDCG@5
BM25-only	0.450	0.550	0.800	0.220	0.546	0.481
Dense-only	0.800	0.900	0.900	0.310	0.842	0.731
Hybrid RRF	0.700	0.800	0.850	0.300	0.752	0.661
Hybrid RRF + Rerank	0.650	0.800	0.850	0.290	0.738	0.664

The important finding is that **Dense-only currently performs strongest on the 20-question retrieval benchmark**. This should not be treated as failure. It is an academically useful finding: for the current cleaned Bangla government-service dataset, semantic retrieval with BGE-M3 is currently more reliable than the initial hybrid/RRF/reranking configuration. The correct next step is not to blindly claim that the advanced method is better, but to tune and evaluate whether hybrid retrieval can improve over this dense baseline.

2. Thesis Objective and System Motivation

The thesis is motivated by a practical problem:

> Bangladeshi government-service information is often distributed across PDFs, HTML pages, notices, scanned documents, tables, FAQs, and mixed Bangla-English content. Users often ask natural Bangla or code-mixed questions, but official information is hard to search, difficult to interpret, and easy to misunderstand.

The system objective is:

> Build a factual, explainable, bilingual Retrieval-Augmented Generation system that answers government-service questions using retrieved evidence rather than unsupported generation.

The major design priorities are:

1. **Accuracy over fluency.**
 2. **Evidence grounding over open-ended generation.**
 3. **Bangla-first retrieval support.**
 4. **Explainability through source IDs and links.**
 5. **Evaluation through measurable retrieval and generation metrics.**
 6. **A modular architecture that can add future domains without rewriting the system.**
-

3. Relationship to the Reference Paper

The reference paper, NextRAG, compares a proposed advanced RAG pipeline against simpler RAG baselines. Our work follows the same research logic, but in a different domain.

3.1 What We Replicate Conceptually

We replicate the following research pattern:

1. Create a domain knowledge base.
2. Build a baseline RAG pipeline.
3. Build a stronger proposed RAG pipeline.
4. Compare both using a fixed question set.
5. Evaluate numerically using retrieval and answer-quality metrics.
6. Analyze performance by question category.
7. Use ablation/parameter tests to justify design choices.

3.2 What Is Different in Our Thesis

Aspect	NextRAG Paper	Civic.ai Thesis
Domain	Financial question answering	Bangladeshi government services
Current data	Financial QA/domain data	Birth and death registration documents
Language focus	Bangla and English	Bangla-first, English and code-mixed support
Data form	Structured QA and domain data	FAQs, legal acts, rules, HTML, notices, converted files, tables
Goal	Financial QA correctness	Civic-service correctness and explainability
User risk	Financial misinformation	Government-service misinformation
Current LLMs	Paper-specific model setup	Local Ollama models: llama3.2 , llama3 , qwen2.5:7b

3.3 Why We Do Not Copy the Paper Blindly

The reference paper is useful as a blueprint, but direct copying would be weak academically because:

- Our domain is not finance.
- Our data is noisier and more document-heavy.
- Bangla government-service documents have legal/procedural wording.
- Exact source grounding is more important than creative answer generation.
- We need to justify design choices using our own experiments.

Therefore, our P2 approach is:

> Replicate the structure of comparison and evaluation, but adapt the pipeline to the government-service domain and validate it using our own data.

4. P2 Objective Mapping

The P2 instruction says:

> Design multiple engineering/theoretical solutions/proof of the problem to meet the desired objectives, needs, and requirements within constraints.

Our mapping:

P2 Requirement	What We Did
Design process or methodology overview	Built a full modular RAG pipeline from raw data to answer generation
Multiple alternative solutions	Dense-only, BM25-only, Hybrid RRF, Hybrid+Rerank, Simple RAG, CivicRAG
Preliminary design/model specification	Defined data structure, chunking strategy, embedding model, vector DB, reranking, generation models
Functional verification/simulation	Ran 20-question retrieval evaluation and 1-question generation/stability smoke test
Constraints	Local LLMs, Bangla text, noisy government data, no API key dependency, explainability
Evidence of design decisions	Reports with Recall@k, MRR, nDCG, Context Precision, generation proxy metrics

This is P2-appropriate because it does not merely present a final system. It demonstrates the **process of selecting and validating design alternatives**.

5. Current Repository and Domain Structure

The repository is designed as a **single multi-domain RAG system**, not separate repositories per service.

```
bilingual-rag-thesis/  
  app.py  
  src/  
    pipeline.py  
    retrieval/  
      hybrid_retriever.py  
    reranking/  
      reranker.py  
    generation/  
      ollama_generator.py  
    evaluation/  
      metrics.py  
  scripts/  
    build_index.py  
    run_p2_experiments.py  
    evaluate_retrieval.py  
    evaluate_generation.py  
  domains/  
    birth_death_registration/  
      config.json  
      data/  
        raw/  
        interim/  
          birth_death_chunks.jsonl  
        processed/  
          chroma_birth_death_bge_m3/  
        evaluation/  
          p2_20q_eval_set.json  
  docs/  
    evaluation/  
      p2_experiments/  
      p2_experiments_smoke_generation/
```

This supports future domains such as:

- domains/passport/
- domains/brta/
- domains/birth_death_registration/
- domains/land_service/
- domains/nid_service/

Each future domain can have its own:

- Raw data.
- Cleaned/interim chunks.
- Chroma collection.
- Config file.
- Evaluation set.

The codebase stays shared.

6. Current End-to-End Pipeline

```
Raw Government Data
↓
Manual / Semi-manual Cleaning
↓
Domain Folder: birth_death_registration
↓
Document-Type-Aware Chunking
↓
Chunk Metadata + Source URLs
↓
Two Text Versions per Chunk
↓
retrieval_text for searching
answer_evidence/content for LLM grounding
↓
Embedding with BAAI/bge-m3
↓
Vector Index in ChromaDB
↓
BM25 Sparse Index
↓
User Query
↓
Language + Intent Detection
↓
Hybrid Retrieval
Dense Search + BM25 Search
↓
RRF Fusion
↓
Reranking / Intent Boosting
↓
Top Evidence Chunks
↓
Safe Answer Path Check
fee answer / correction answer / procedural answer
↓
If Safe Path Exists
Controlled Grounded Answer
↓
If No Safe Path Exists
Local LLM Generation
llama3.2 / llama3 / qwen2.5:7b
↓
Bangla Language Guard
↓
Final Answer
↓
Source IDs + Retrieval Scores + Official Links
```

Why This Pipeline Was Chosen

Pipeline Step	Why It Exists
Manual/semi-manual cleaning	Government documents contain web noise, OCR artifacts, duplicated navigation text, and inconsistent formatting
Document-type-aware chunking	FAQs, legal rules, fee tables, and process pages require different chunking logic
Metadata	Allows filtering, debugging, source display, category analysis, and future evaluation
<code>retrieval_text</code> vs <code>content</code>	Search can use aliases and query-friendly terms, while LLM sees original grounded evidence
BGE-M3 embeddings	Strong multilingual embedding candidate with Bangla support
ChromaDB	Local persistent vector index suitable for thesis prototype
BM25	Captures exact legal/service terms, numbers, fees, and Bangla keywords
RRF	Combines dense semantic retrieval and lexical retrieval
Reranking	Attempts to improve candidate ordering before generation
Safe answer paths	Avoids hallucination for fee/correction/procedure answers
Local LLMs	Avoids API key dependency and supports reproducible local experiments
Source IDs and links	Makes answers explainable and manually verifiable

7. Data Preparation and Chunking

7.1 Raw Data Types Considered

The birth/death registration domain included four broad raw data types:

1. Bijoy-encoded PDFs.
2. Converted DOCX files.
3. Unicode HTML pages.
4. Scanned PDFs / OCR-related files.

These are realistic government-service data formats. They are not clean tutorial-style documents.

7.2 Why Manual Cleaning Was Necessary

Manual cleaning was necessary because many source pages contained:

- Website navigation menus.
- Footer text.

- Repeated service lists.
- Accessibility controls.
- News blocks.
- Minister/officer profile sections.
- Duplicate sidebar content.
- Mixed useful and irrelevant text.
- Table-like content copied as plain text.

If these noisy blocks are embedded, the retriever may return irrelevant chunks.

7.3 Chunking Strategy

The current chunking approach is **document-type-aware**:

Document Type	Chunking Strategy
FAQ JSON	One question-answer pair per chunk
FAQ Markdown	Heading/question-based chunking
Legal rules/acts	Section/rule-based chunking
Application process	Step/heading-based chunks
Fee tables	Table-level and row-level chunks
Notice/OCR documents	Heading and semantic block chunks

This is better than using one generic fixed-size splitter for all documents because:

- FAQ answers should not be split away from their questions.
- Legal rules should preserve rule numbers and legal context.
- Fee rows need exact row-level retrieval.
- Process steps should remain ordered and explainable.

7.4 Important Chunk File

The final chunk file used for indexing is:

```
domains/birth_death_registration/data/interim/birth_death_chunks.jsonl
```

Each chunk contains:

- `id`
- `content`
- `retrieval_text`
- `metadata`

- source/link fields where available

7.5 Why We Use Two Text Versions

Each chunk has two conceptual versions:

1. **retrieval_text**

- Used for search.
- Can contain aliases, normalized wording, and query-friendly terms.
- Helps cross-lingual or paraphrased matching.

1. **content** / **answer evidence**

- Used for answer generation.
- Should preserve original Bangla source wording.
- Prevents the LLM from answering based on artificial aliases.

This design reduces hallucination because the LLM receives source evidence rather than search-only expansion text.

8. Embedding and Indexing

8.1 Embedding Model

Current embedding model:

BAAI/bge-m3

Reason for choosing BGE-M3:

- Multilingual support.
- Strong semantic retrieval capability.
- Suitable for Bangla-English/code-mixed retrieval.
- Can run locally.
- Works with ChromaDB cosine similarity.

8.2 Vector Database

Current vector database:

ChromaDB

Reason:

- Local persistent database.

- No cloud/API dependency.
- Simple to inspect and rebuild.
- Sufficient for thesis-scale prototype.

8.3 Sparse Index

Current sparse retriever:

```
BM25Okapi
```

Reason:

- Captures exact lexical matches.
- Useful for legal terms, fees, dates, rule numbers, and exact service names.
- Provides a strong non-neural baseline.

9. Retrieval Variants Implemented

The retrieval comparison is implemented in:

```
scripts/run_p2_experiments.py
```

The core variant function is:

```
def retrieval_variants(pipeline, query, top_k):
    hybrid_candidates = pipeline.retriever.search(...)
    return {
        "dense_only": pipeline.retriever.dense_only_search(query, top_k=top_k),
        "bm25_only": pipeline.retriever.bm25_only_search(query, top_k=top_k),
        "hybrid_rrf": hybrid_candidates[:top_k],
        "hybrid_rrf_rerank": pipeline.reranker.rerank(query, hybrid_candidates,
top_k=top_k),
    }
```

9.1 Dense-only

Dense-only means:

```
Query → BGE-M3 embedding → ChromaDB cosine similarity → top-k chunks
```

It captures semantic similarity. Example:

```
Query: "জন্মসনদ হারিয়ে গেলে কী করব?"
```

Dense retrieval can match:

"সন্দের প্রতিলিপি", "নষ্ট", "হারানো", "নিবন্ধকের কাছে আবেদন"

even if exact query words are not identical.

9.2 BM25-only

BM25-only means:

Query tokens → lexical score against chunk tokens → top-k chunks

BM25 is useful when exact terms matter:

ফি, বিধি ১৩, জন্ম তারিখ, প্রতিলিপি, ৫০ টাকা, ৪৫ দিন

However, BM25 can struggle with:

- Bangla spelling variation.
- OCR noise.
- Code-mixed queries.
- Paraphrases.
- Synonyms.

9.3 Hybrid RRF

Hybrid RRF means:

Dense ranked list + BM25 ranked list → Reciprocal Rank Fusion → fused top-k chunks

RRF does not only select chunks common to both lists. It combines rankings:

$$\text{RRF_score}(d) = \sum \text{weight_r} / (k + \text{rank_r}(d))$$

Where:

- d = document/chunk.
- r = retriever, e.g., dense or BM25.
- $\text{rank_r}(d)$ = rank of chunk d in retriever r .
- k = RRF smoothing constant.
- weight_r = retriever weight.

If a chunk appears in both Dense and BM25, it receives both contributions. If it appears in only one, it can still survive if ranked high.

9.4 Hybrid + Rerank

Hybrid + Rerank means:

```
Dense + BM25 → RRF candidate pool → reranker → final top-k chunks
```

In the full pipeline, reranking can use a cross-encoder model. In the fast P2 experiment run, we used lexical fallback mode to avoid long CPU runtime:

```
Reranking mode: lexical fallback / fast mode
```

This means the current fast results are preliminary and practical for P2 iteration. For final thesis results, we should also run full cross-encoder reranking where feasible.

10. Simple RAG vs CivicRAG

10.1 Simple RAG

Simple RAG is our tutorial/baseline system:

```
Query → Dense retrieval → top chunks → LLM answer
```

It is useful because:

- It is easy to explain.
- It provides a minimal baseline.
- It shows whether advanced engineering actually improves results.

10.2 CivicRAG

CivicRAG is our proposed system:

```
Query normalization
↓
Intent detection
↓
Hybrid retrieval
↓
RRF fusion
↓
Reranking/boosting
↓
Safe answer paths
↓
LLM only if needed
↓
Bangla language guard
↓
Sources and links
```

CivicRAG is not just a different LLM. It is an engineered RAG pipeline with stronger retrieval, routing, grounding, answer controls, and explainability.

11. Evaluation Dataset

The P2 evaluation dataset is:

```
domains/birth_death_registration/data/evaluation/p2_20q_eval_set.json
```

It contains 20 Bangla scenario-based questions. Each record contains:

```
{
  "id": "...",
  "category": "...",
  "question_type": "...",
  "question": "...",
  "expected_answer": "...",
  "expected_source_ids": [...]
}
```

Why Scenario-Based Questions?

Simple binary questions are useful but insufficient. Real citizens ask messy questions such as:

আমার ছেলের জন্য জন্ম নিবন্ধন আবেদন করেছি। আবেদন নাম্বার পেয়েছি কিন্তু কতদিন সময় লাগবে বুঝছি না।
ফাইল upload হচ্ছে কিন্তু submit করার সময় প্রয়োজনীয় ফাইল আপলোড করেননি দেখাচ্ছে।
আমার ভোটার আইডি আছে কিন্তু জন্ম নিবন্ধন অনলাইন করা নাই, এখন কী করব?

This requires:

- Multi-intent detection.

- Retrieval from multiple chunks.
- Avoiding unsupported claims.
- Explaining what the dataset does and does not contain.

Categories Included

The 20 questions include:

- Application process.
- Required documents.
- Lost certificate.
- Single-parent/divorce scenarios.
- Online record not found.
- Upload/file attachment issues.
- Manual-to-online migration.
- Overseas registration.
- Parent name correction.
- Birth date correction.
- Fee/fee waiver.
- Death registration.
- Out-of-context questions.

This aligns with the P2 goal of testing multiple cases and constraints rather than only demonstrating happy-path queries.

12. Evaluation Metrics and Exact Formulas

The current metric implementation is in:

```
src/evaluation/metrics.py
```

Let:

- G = set of expected relevant source chunk IDs for a question.
- R_k = top-k retrieved chunk IDs.
- $\text{rank}(i)$ = rank position of retrieved chunk i .

12.1 Recall@k

Current implementation:

$\text{Recall@k} = 1$ if any retrieved chunk in top-k is in `expected_source_ids`
 $\text{Recall@k} = 0$ otherwise

Formula:

$\text{Recall@k} = 1 [R_k \cap G \neq \emptyset]$

Interpretation:

$\text{Recall@5} = 0.850$

means:

17 out of 20 questions retrieved at least one expected source chunk within the top 5.

This is an RAG-style evidence-hit recall. It answers:

> Did the retriever bring at least one correct evidence chunk into the context window?

12.2 Context Precision@k

Formula:

$\text{Context Precision@k} = |R_k \cap G| / k$

Interpretation:

If top 5 contains 2 expected chunks:

$\text{Context Precision@5} = 2 / 5 = 0.4$

This tells us how much of the retrieved context is actually useful.

12.3 MRR

MRR means Mean Reciprocal Rank.

For one question:

$RR = 1 / \text{rank_of_first_relevant_chunk}$

If the first relevant chunk is rank 1:

$RR = 1 / 1 = 1.0$

If it is rank 4:

$$RR = 1 / 4 = 0.25$$

Across all questions:

$$MRR = \text{average}(RR \text{ over all questions})$$

MRR is important because two systems can both have Recall@5 = 1, but one may rank the evidence first while another ranks it fifth.

12.4 nDCG@k

nDCG evaluates ranking quality.

DCG:

$$DCG@k = \sum \text{relevance}_i / \log_2(i + 1)$$

For binary relevance:

```
relevance_i = 1 if retrieved chunk at rank i is expected  
relevance_i = 0 otherwise
```

IDCG is the ideal DCG if all relevant chunks were ranked at the top.

$$nDCG@k = DCG@k / IDCG@k$$

Interpretation:

- Higher nDCG means relevant chunks appear earlier.
- It is stronger than Recall@k because it considers ranking order.

12.5 Token F1

Used for generation comparison:

```
Precision = overlapping_tokens / generated_tokens  
Recall = overlapping_tokens / reference_tokens  
F1 = 2 × Precision × Recall / (Precision + Recall)
```

This is only a rough proxy for Bangla answer correctness.

12.6 Semantic Correctness Proxy

We encode generated answer and expected answer using the embedding model:

```
semantic_correctness = cosine(embedding(answer), embedding(expected_answer))
```


Cosine similarity:

```
cosine(a,b) = (a · b) / (||a|| ||b||)
```

This is not perfect but helps compare answer meaning.

12.7 Faithfulness Proxy

Current lightweight proxy:

```
faithfulness_proxy = max cosine(answer_embedding, context_chunk_embedding)
```

This estimates whether the answer is semantically close to retrieved evidence.

Important limitation:

> This is not a full RAGAS faithfulness judge. It is a fast local proxy. For final thesis, we should combine it with manual review and possibly RAGAS/LangSmith style evaluation.

12.8 Stability Metrics

For repeated query tests:

```
Exact Answer Match = 1 if repeated answers are exactly the same  
Mean Pairwise Answer Similarity = average cosine similarity between repeated answers  
Mean Source Jaccard = |S_i ∩ S_j| / |S_i ∪ S_j|
```

In government-service QA, identical answers are not automatically bad. Stable factual answers can be desirable if they are grounded and correct.

13. Current Experiment Results

13.1 Retrieval Variant Comparison

Command used:

```
.venv/bin/python scripts/run_p2_experiments.py --fast-rerank
```

Output report:

```
docs/evaluation/p2_experiments/p2_experiment_report.md
```

Results:

Variant	Recall@1	Recall@3	Recall@5	Context Precision@5	MRR	nDCG@5
BM25-only	0.450	0.550	0.800	0.220	0.546	0.481
Dense-only	0.800	0.900	0.900	0.310	0.842	0.731
Hybrid RRF	0.700	0.800	0.850	0.300	0.752	0.661
Hybrid RRF + Rerank	0.650	0.800	0.850	0.290	0.738	0.664

13.2 Interpretation

Dense-only currently performs best across:

- Recall@1.
- Recall@3.
- Recall@5.
- Context Precision@5.
- MRR.
- nDCG@5.

This means the BGE-M3 embedding model is currently retrieving relevant chunks very well for our cleaned Bangla dataset.

13.3 Does This Mean the Dataset Is Bad?

No. It may actually mean the dataset is clean enough for semantic retrieval to work well.

Possible reasons dense-only is strong:

- Chunks are semantically coherent.
- The dataset is domain-focused.
- BGE-M3 handles Bangla semantic similarity well.
- Many user questions are paraphrased/scenario-based rather than exact keyword queries.

Possible reasons hybrid currently underperforms:

- BM25 may introduce keyword-matched but semantically weaker chunks.
- RRF weights may give BM25 too much influence.
- Fast rerank used lexical fallback instead of full cross-encoder reranking.
- Some expected-source labels may favor semantic chunks over lexical chunks.
- Bangla tokenization in BM25 is basic.

The correct conclusion is:

> Dense-only is currently the strongest baseline. Hybrid/RRF needs tuning before being claimed as the final best retrieval design.

This is academically healthy because it shows the system is being selected based on results rather than assumptions.

14. Parameter Sensitivity Results

Current fast parameter sensitivity results:

Config	Recall@1	Recall@3	Recall@5	Context Precision@5	MRR	nDCG@5
Balanced current	0.650	0.800	0.850	0.290	0.738	0.664
BM25-heavy	0.650	0.800	0.800	0.300	0.717	0.642
Dense-heavy	0.650	0.800	0.850	0.290	0.735	0.663
Large top-k	0.650	0.800	0.800	0.290	0.717	0.658
Small top-k	0.650	0.850	0.850	0.290	0.742	0.663

Interpretation:

- BM25-heavy does not improve overall recall.
- Smaller candidate pools sometimes improve Recall@3 slightly.
- Current RRF settings are not yet beating dense-only.
- We need more systematic tuning before finalizing hybrid.

15. Generation and Stability Smoke Test

A small smoke test was run:

```
.venv/bin/python scripts/run_p2_experiments.py \  
  --limit 1 \  
  --fast-rerank \  
  --run-generation \  
  --run-stability \  
  --stability-repeats 2 \  
  --models llama3.2 \  
  --methods simple civic \  
  --answer-timeout-seconds 45 \  
  --output-dir docs/evaluation/p2_experiments_smoke_generation
```

Results:

Method + Model	Semantic Correctness	Token F1	Answer Relevance	Faithfulness Proxy	Source Retrieved	Source Cited
CivicRAG + llama3.2	0.852	0.158	0.726	0.809	1.000	1.000
Simple RAG + llama3.2	0.000	0.000	0.000	0.000	0.000	0.000

Important caveat:

- The Simple RAG answer timed out in this smoke test and was scored as zero.
- This does not prove Simple RAG is always bad.
- It only confirms that the evaluation framework can run generation comparison, timeout safely, and record errors.

Stability result for CivicRAG + llama3.2:

Metric	Value
Exact Answer Match	1.000
Mean Pairwise Answer Similarity	1.000
Mean Source Jaccard	1.000

Interpretation:

- For this one test question, CivicRAG gave stable repeated answers.
- In civic QA, stability is generally positive if the answer is factually correct and source-grounded.

16. How to Explain This Without Saying "We Used AI"

A strong faculty-facing explanation:

> We implemented an evaluation harness that runs the same manually prepared Bangla test questions through multiple retrieval strategies. For each question, we manually assigned expected relevant source chunk IDs. The system retrieves top-k chunks using each strategy, compares retrieved chunk IDs with expected source IDs, and calculates Recall@k, Context Precision@k, MRR, and nDCG@k. This lets us compare design alternatives quantitatively before selecting the final retrieval architecture.

Do not say:

AI compared them for us.

Say:

We designed a reproducible retrieval evaluation framework and used it to measure alternative retrieval strategies.

17. How to Demo the System

17.1 Start the Chatbot

```
.venv/bin/python app.py --host 127.0.0.1 --port 7860
```

Open:

```
http://127.0.0.1:7860
```

17.2 Run Retrieval Evaluation

Fast retrieval evaluation:

```
.venv/bin/python scripts/run_p2_experiments.py --fast-rerank
```

View:

```
docs/evaluation/p2_experiments/p2_experiment_report.md
```

17.3 Run Small Generation Smoke Test

```
.venv/bin/python scripts/run_p2_experiments.py \  
  --limit 1 \  
  --fast-rerank \  
  --run-generation \  
  --run-stability \  
  --stability-repeats 2 \  
  --models llama3.2 \  
  --methods simple civic \  
  --answer-timeout-seconds 45 \  
  --output-dir docs/evaluation/p2_experiments_smoke_generation
```

17.4 Suggested Demo Questions

Use questions that show different capabilities:

1. জন্মনিবন্ধন সনদ হারিয়ে গিয়েছে, কি করব?
2. জন্মসনদের জন্য আবেদন করব কীভাবে? কি কি কাগজপত্র প্রয়োজন?
3. ডিভোর্সের ক্ষেত্রে, শুধু পিতার/মাতার তথ্য দিয়ে কি সন্তানের জন্ম নিবন্ধন করা যাবে?

4. আমার জন্ম নিবন্ধন অনলাইনে খুঁজে পাচ্ছি না, কী করব?
5. আমার ছেলের জন্ম নিবন্ধনে বাবা-মায়ের নাম ভুল আছে, কীভাবে সংশোধন করব?
6. এতিম শিশুর জন্ম নিবন্ধনে ফি লাগবে কি?
7. পাসপোর্ট করতে কী লাগে?

The final question is intentionally out-of-context. The system should avoid answering passport questions from birth/death registration data.

18. Why Dense-only Being Better Is Not a Bad Sign

It is natural to feel worried when an advanced method performs worse than a simpler method. But in research, this is exactly why baselines exist.

Possible interpretations:

Interpretation 1: The Dataset Is Clean and Semantic

Because we cleaned and chunked the data carefully, semantic retrieval may already work well.

This is positive.

Interpretation 2: BM25 Needs Better Bangla Tokenization

Current BM25 tokenization is simple. Bangla morphology and spelling variation may reduce lexical matching quality.

This is fixable.

Interpretation 3: RRF Weights Need Tuning

Current RRF weights:

```
dense = 0.55  
bm25 = 0.45
```

Maybe BM25 should be lower, such as:

```
dense = 0.80  
bm25 = 0.20
```

Interpretation 4: Hybrid May Help Specific Categories Only

Overall average may favor dense-only, but BM25/hybrid may help:

- Fee questions.
- Legal rule number questions.
- Exact date or amount questions.

- Service-code questions.

Therefore, we should calculate category-wise metrics.

Interpretation 5: Fast Rerank Is Not Final Rerank

The current report used:

```
lexical fallback / fast mode
```

The final thesis should also test:

```
full BGE cross-encoder reranker
```

if runtime allows.

19. Recommended Next Steps

Short-Term P2 Next Steps

1. Add category-wise evaluation tables.
2. Compare dense-only vs hybrid by question category.
3. Try denser RRF weights:

- `dense=0.70` , `bm25=0.30`
- `dense=0.80` , `bm25=0.20`
- `dense=0.90` , `bm25=0.10`

1. Run full cross-encoder reranking on a smaller subset.
2. Add manual qualitative scoring for the 20-question set.

Medium-Term Thesis Steps

1. Expand evaluation set to 50+ Bangla/code-mixed questions.
2. Add answer-level manual grading:

- Correctness.
- Faithfulness.
- Completeness.
- Language match.
- Source usefulness.

1. Evaluate all three local LLMs:

- llama3.2
- llama3
- qwen2.5:7b

1. Compare LLM-only answers against RAG answers.
2. Add out-of-context refusal metrics.
3. Decide final retrieval architecture based on measured results.

20. Alternative Designs Considered

Alternative	Why Considered	Why Not Final Yet
Dense-only	Strong semantic baseline, simple, currently best retrieval result	May miss exact legal/fee terms
BM25-only	Strong exact-match baseline, interpretable	Weak on paraphrased Bangla/code-mixed queries
Hybrid RRF	Combines semantic and lexical evidence	Current weights not beating dense-only
Hybrid + Rerank	Should improve ranking of candidate chunks	Fast rerank underperformed; full reranker needs runtime testing
LLM-only	Simple to build	High hallucination risk, no grounded evidence
Translation-first pipeline	Could translate Bangla to English then retrieve	Risky for legal/government terms and adds translation errors
Fine-tuning LLM	Could improve language style	Not needed before retrieval quality is solved; costly and less explainable
Knowledge graph	Useful for structured legal/service relations	Too heavy for current P2; possible future extension
Agentic RAG	Useful for complex workflows	Could overcomplicate current phase; core retrieval must be validated first

21. Supervisor Questions and Prepared Answers

Q1. What exactly is your proposed system?

Answer: Our proposed system is CivicRAG, a Bangla-English government-service RAG pipeline. It retrieves evidence from cleaned government-service documents, ranks the evidence, applies safe answer logic for procedural/fee/correction questions, and uses local LLMs only when necessary. It returns answers with source IDs and official links.

Q2. Is your model one of llama3.2, llama3, or qwen2.5?

Answer: No. Those are backend LLMs. Our contribution is the RAG system architecture and retrieval/generation pipeline. We evaluate CivicRAG with different local LLM backends to see which one produces better grounded answers.

Q3. Why are you comparing Dense-only, BM25-only, Hybrid, and Hybrid+Rerank?

Answer: These are alternative retrieval designs. P2 requires multiple engineering solutions and functional verification. We compare them using the same 20-question gold set to decide which retrieval design is most suitable for the final system.

Q4. How did you calculate Recall@5?

Answer: For each question, we have manually assigned expected source chunk IDs. If any expected source appears in the top 5 retrieved chunks, that question gets Recall@5 = 1. Otherwise it gets 0. The reported Recall@5 is the average across all questions. So 0.850 means 17 out of 20 questions retrieved at least one expected source in top 5.

Q5. Why is Dense-only currently better than Hybrid?

Answer: The current dataset is semantically clean and BGE-M3 performs well for Bangla semantic matching. BM25 may introduce keyword-matched but less relevant chunks, so the initial hybrid weighting is not yet optimal. This does not mean hybrid is wrong; it means we need tuning and category-wise analysis before choosing the final retrieval setup.

Q6. Does this mean your proposed CivicRAG is worse?

Answer: Not necessarily. The current table evaluates retrieval variants only. CivicRAG includes more than retrieval: intent detection, safe answer paths, source links, language guard, and grounded answer formatting. However, retrieval quality is the foundation, so we must tune the retrieval component honestly.

Q7. Why not just use Dense-only as final?

Answer: Dense-only may become the final retrieval choice if it consistently wins. But before deciding, we need category-wise evaluation. BM25/hybrid may be better for exact legal, fee, rule-number, or date-based questions. Final selection should be evidence-based.

Q8. Why use BGE-M3?

Answer: BGE-M3 is multilingual and suitable for Bangla-English retrieval. It supports semantic retrieval better than English-only embedding models and can run locally, matching our no-API constraint.

Q9. Why use BM25 if it performs worse?

Answer: BM25 is still valuable as a baseline and may help exact-match queries. Even if not final, including it makes the study academically stronger because we compare neural retrieval against a

classical IR method.

Q10. Why use RRF?

Answer: RRF is a simple, explainable rank-fusion method. It combines dense and sparse rankings without needing score calibration. Chunks appearing high in both lists receive stronger scores.

Q11. Does RRF only keep common results?

Answer: No. RRF rewards common results but does not require overlap. A chunk appearing in only Dense or only BM25 can still remain if ranked high enough.

Q12. What is the formula for RRF?

Answer:

$$\text{RRF}(d) = \sum \text{weight}_r / (k + \text{rank}_r(d))$$

Where `d` is a chunk, `r` is a retriever, and `rank_r(d)` is that chunk's rank in retriever `r`.

Q13. Why not use only LLMs without retrieval?

Answer: LLM-only answers can hallucinate and cannot provide source grounding. Government-service information requires factual correctness and source traceability, so retrieval is necessary.

Q14. Why local LLMs?

Answer: Local LLMs avoid API dependency, reduce cost, and make the thesis reproducible. We use Ollama models such as `llama3.2`, `llama3`, and `qwen2.5:7b`.

Q15. Why do some answers look identical across models?

Answer: In our system, many answers are generated through controlled safe paths from retrieved evidence, not fully free-form LLM generation. For civic QA, factual consistency is often positive. Variation is less important than correctness and grounding.

Q16. How do you handle Bangla queries?

Answer: The pipeline detects Bangla text, uses Bangla-compatible embeddings, retrieves Bangla evidence, and applies a Bangla language guard so Bangla questions should receive Bangla answers.

Q17. What happens if the question is outside the domain?

Answer: The system should avoid answering from unrelated sources. Out-of-context questions are included in the evaluation set to test refusal behavior.

Q18. How do you know the system is grounded?

Answer: Each answer includes source IDs and official/source links. We also evaluate whether expected source IDs were retrieved and cited. Manual review is still needed for final faithfulness judgment.

Q19. Are your metrics enough?

Answer: For P2, retrieval metrics plus generation proxy metrics are enough to demonstrate design verification. For final thesis, we should add manual qualitative analysis and possibly RAGAS/LangSmith-style faithfulness evaluation.

Q20. What is your current limitation?

Answer: Current limitations include a small 20-question evaluation set, fast rerank mode instead of full reranker in the latest report, limited manual answer grading, and a single active domain. These are planned next steps.

22. Risks and Mitigation

Risk	Mitigation
Hybrid underperforms dense-only	Treat dense-only as strong baseline; tune RRF weights; run category-wise analysis
LLM hallucination	Use safe answer paths and source-grounded prompts
Bangla output inconsistency	Use language guard and Bangla evidence
Slow local LLMs	Add generation timeout and smoke tests
Evaluation dataset too small	Expand from 20 to 50+ questions
Manual labels may be incomplete	Review expected source IDs and add multiple valid source chunks
BM25 weak for Bangla	Improve tokenization or use sparse multilingual retriever later
Reranker slow on CPU	Use fast mode for iteration and full mode for selected final runs

23. What We Can Claim Now

We can safely claim:

1. We have built a modular birth/death registration RAG prototype.
2. The system supports local LLM backends.
3. The system uses cleaned, chunked, metadata-rich Bangla government-service data.
4. We implemented multiple retrieval design alternatives.
5. We built a reproducible evaluation harness.
6. We measured retrieval performance using Recall@k, Context Precision, MRR, and nDCG.
7. Initial retrieval results show Dense-only is currently the strongest baseline.

8. The proposed CivicRAG architecture includes additional grounding and safety mechanisms beyond retrieval alone.
9. Further tuning is required before claiming the advanced hybrid pipeline is final.

We should not yet claim:

1. Hybrid is definitely better than Dense-only.
 2. The system is fully production-ready.
 3. The chatbot has been evaluated on all government-service domains.
 4. Automatic metrics alone prove answer correctness.
 5. LLM-generated answers are always faithful.
-

24. Final P2 Narrative

A clean way to present P2:

> In Phase 2, we designed a modular RAG architecture for Bangladeshi government-service question answering, focusing first on birth and death registration. We processed noisy government documents into structured chunks with metadata and source links. We implemented multiple retrieval alternatives: Dense-only, BM25-only, Hybrid RRF, and Hybrid+Rerank. We then built a 20-question Bangla scenario-based evaluation set with manually selected expected source chunks. Using Recall@k, MRR, nDCG, and Context Precision, we compared the alternatives. The first result showed that Dense-only retrieval currently performs best, which indicates that BGE-M3 embeddings are strong for this cleaned Bangla dataset. Hybrid retrieval remains a candidate but requires further tuning. This process satisfies the P2 objective because it demonstrates design alternatives, functional verification, measurable evaluation, and evidence-based design selection.

25. Appendix: Key Files

Purpose	File
Chatbot app	app.py
Main pipeline	src/pipeline.py
Retriever	src/retrieval/hybrid_retriever.py
Reranker	src/reranking/reranker.py
LLM generator	src/generation/ollama_generator.py
Evaluation metrics	src/evaluation/metrics.py
P2 experiment runner	scripts/run_p2_experiments.py
Domain config	domains/birth_death_registration/config.json
Final chunk file	domains/birth_death_registration/data/interim/birth_death_chunks.jsonl
P2 eval set	domains/birth_death_registration/data/evaluation/p2_20q_eval_set.json
Retrieval report	docs/evaluation/p2_experiments/p2_experiment_report.md
Smoke generation report	docs/evaluation/p2_experiments_smoke_generation/p2_experiment_report.md

26. Appendix: Recommended Slide Outline

1. Problem statement.
2. Dataset and domain scope.
3. Challenges in Bangla government-service data.
4. Proposed modular RAG architecture.
5. Chunking and metadata design.
6. Retrieval alternatives.
7. Evaluation dataset.
8. Metric formulas.
9. Current results.
10. Interpretation: why dense-only currently wins.
11. CivicRAG beyond retrieval.
12. Demo.
13. Limitations and next steps.